Kairos - Sistema Distribuído de Despacho de Emergências

Universidade Europeia / IADE - Engenharia Informática
Projeto Multidisciplinar - 5º semestre (2026/2027)
Grupo 08
Elementos do Grupo: Pedro António - 20241273, Mateus Reis - 20241799, Francisco Abecasis - 20240120, Deolindo Soares - 20221446

GitHub: Repositório GitHub
Trello: Trello



Palavras-chave

sistemas distribuídos, tolerância a faltas, despacho de emergências, Kubernetes, replicação, problemas de satisfação de restrições, A*, tempo real

1. Introdução

Este relatório apresenta a primeira versão do projeto **Kairos**, desenvolvido no âmbito do projeto multidisciplinar de 5.º semestre da Licenciatura em Engenharia Informática.

A **Kairos** é uma plataforma web de despacho de emergências que recebe ocorrências (emergências médicas, incêndios, acidentes), acompanha em tempo real a posição e o estado das unidades de socorro e **atribui automaticamente a unidade mais adequada a cada ocorrência**, calculando a rota mais rápida até ao local.

Por se tratar de um sistema crítico, a plataforma é construída sobre uma **arquitetura distribuída e tolerante a faltas**: todos os componentes estão replicados, e a falha de um servidor, de uma réplica da base de dados ou de uma máquina inteira não interrompe o serviço nem provoca a perda de ocorrências.

Esta primeira entrega centra-se na **conceção do sistema**: definição do problema, requisitos, arquitetura distribuída, **modelo de faltas**, componentes de IA e segurança, e planeamento do desenvolvimento.

2. Metodologia

O projeto segue uma **metodologia ágil baseada em Kanban**. O trabalho flui de forma contínua por um quadro de tarefas, em vez de ser organizado em ciclos fechados, o que se adapta melhor a uma equipa com disponibilidades variáveis ao longo do semestre. Para garantir o cumprimento das três entregas, o quadro é complementado por **milestones** com datas fixas.

O quadro é gerido no **Trello** o que garante a rastreabilidade de todo o trabalho realizado.

2.1 Quadro Kanban

Coluna	Significado
Backlog	Tarefas identificadas, ainda não priorizadas
To Do	Tarefas priorizadas para o marco atual, prontas a ser iniciadas
Doing	Tarefas em desenvolvimento, com um responsável atribuído
Blocked	Tarefa bloqueada, espera por ajuda de outro elemento ou finalização de outra tarefa
In Review	Tarefa concluída, espera por testes e revisão por outro elemento
Done	Tarefa revista e integrada no ramo principal

2.2 Regras de Trabalho

Regra	Descrição
Limite de trabalho em curso (WIP)	Cada elemento tem no máximo 2 tarefas em "Doing" em simultâneo
Uma tarefa, um cartão	Todo o trabalho é registado como cartões no Trello, com responsável e etiquetas
Definição de Done	Uma tarefa só passa a "Done" quando o código ou documento está revisto, integrado e testado
Ponto de situação semanal	Reunião curta com o quadro como base: o que avançou, o que está bloqueado e o que se segue

2.3 Etiquetas

Tipo	Etiquetas
Natureza da tarefa	`feature`, `bug`, `docs`, `teste`
Prioridade	`alta`, `média`, `baixa`

2.4 Marcos (Milestones)

Marco	Data	Objetivo
Entrega 1 – Conceção	11/10/2026	Problema, requisitos, arquitetura, modelo de faltas, relatório v1
Entrega 2 – Protótipo	15/11/2026	Cluster Kubernetes, backend replicado, CSP e A* funcionais, primeiros testes de falha, relatório v2
Entrega 3 – Versão final	20/12/2026	Frontend completo, segurança, testes de falha automatizados, demonstração, relatório final

Cada marco agrupa os cartões que têm de estar concluídos até à respetiva data, permitindo acompanhar no Trello a percentagem de trabalho concluído em cada entrega.

Este relatório corresponde ao marco **Entrega 1**.

---

3. Descrição e Problema

Numa emergência, o tempo até à chegada do socorro é determinante. Em situações como uma paragem cardiorrespiratória, cada minuto de atraso reduz de forma significativa as hipóteses de sobrevivência, e parte desse tempo é gasto **antes de a unidade sair** a decidir que meio enviar e por onde.

Em muitos cenários esta decisão depende do operador, que escolhe a unidade com base na experiência e numa lista de meios disponíveis. Esta abordagem apresenta três problemas:

Problema	Consequência
O operador não sabe com exatidão **quanto tempo** cada unidade demora a chegar	A unidade mais próxima em linha reta pode não ser a mais rápida pelas ruas
As ocorrências são tratadas por **ordem de chegada**	Uma ocorrência leve pode ficar com a unidade de que uma ocorrência crítica vai precisar logo a seguir
Não existe **visão do conjunto**	Uma zona pode ficar sem unidades disponíveis sem que ninguém se aperceba

A estes problemas soma-se um requisito técnico: uma central de emergência **não pode parar**. Se o servidor que a suporta falhar, as ocorrências deixam de ser registadas e atribuídas. Um sistema centralizado num único servidor representa um ponto único de falha inaceitável neste contexto.

A **Kairos** pretende resolver ambos os problemas: **decidir melhor**, através de Inteligência Artificial, e **nunca parar**, através de uma arquitetura distribuída e tolerante a faltas.

---

4. Objetivos & Motivação

4.1 Objetivos

- Desenvolver uma plataforma web responsiva de despacho de emergências.
- Registar ocorrências e acompanhar unidades de socorro num mapa em tempo real.
- Atribuir automaticamente unidades a ocorrências através de **CSP**, com reatribuição quando surge uma ocorrência mais grave.
- Calcular rotas pela rede de estradas real através do algoritmo **A***.
- Garantir que **nenhuma ocorrência confirmada é perdida**, mesmo perante a falha de componentes.
- Manter o serviço disponível perante a falha de qualquer componente individual ou de uma máquina inteira.
- Demonstrar experimentalmente a tolerância a faltas através de testes de falha.

4.2 Motivação

Os sistemas de despacho são um exemplo claro de sistema onde a **distribuição é necessária**, a disponibilidade e a integridade dos dados têm impacto direto na vida das pessoas. Este contexto permite trabalhar, de forma justificada, todos os conceitos centrais de sistemas distribuídos (replicação, consistência, deteção de falhas e recuperação), ao mesmo tempo que oferece problemas reais para as restantes UCs.

---

5. Atores

Ator	Descrição	Necessidade principal
Operador da central	Recebe chamadas e regista ocorrências, supervisiona o estado de todas as unidades	Ver rapidamente que unidade enviar e confirmar a atribuição
Unidade de socorro	Tripulação de ambulância, bombeiros ou polícia	Receber a missão, a rota e atualizar o seu estado (a caminho, no local,

Ator	Descrição	Necessidade principal
		disponível)
Administrador	Gere utilizadores, unidades e bases	Configurar o sistema e consultar o registo de auditoria

6. Pesquisa de mercado

Os sistemas de despacho assistido por computador (**CAD – Computer-Aided Dispatch**) são utilizados por serviços de emergência em todo o mundo. Em Portugal, as chamadas de emergência médica são encaminhadas para os **CODU** (Centros de Orientação de Doentes Urgentes) do INEM.

Solução	Pontos fortes	Limitações	Oportunidade para a Kairos
Sistemas CAD comerciais	Completos, integrados com comunicações rádio e telefone	Soluções proprietárias, de custo elevado e pouco transparentes	Demonstrar os mecanismos de decisão e de tolerância a faltas de forma aberta
Despacho manual com mapa	Simples, controlado pelo operador	Decisão dependente da experiência. Sem estimativa de tempos reais	Proposta automática baseada em tempos de chegada calculados
Atribuição pela unidade mais próxima	Rápida e fácil de implementar	Decisão isolada por ocorrência, ignora prioridades e cobertura	Decisão conjunta de várias ocorrências, com reatribuição por prioridade

A **Kairos** diferencia-se por combinar, num só sistema:

1. **Atribuição otimizada** de várias ocorrências em simultâneo, com restrições explícitas (CSP).
2. **Reatribuição dinâmica** de unidades em trânsito quando surge uma ocorrência mais grave.
3. **Tempos de chegada reais**, calculados pela rede de estradas (A*).
4. **Tolerância a faltas demonstrável**, com garantias explícitas sobre disponibilidade e perda de dados.

7. Requisitos

7.1 Requisitos Funcionais

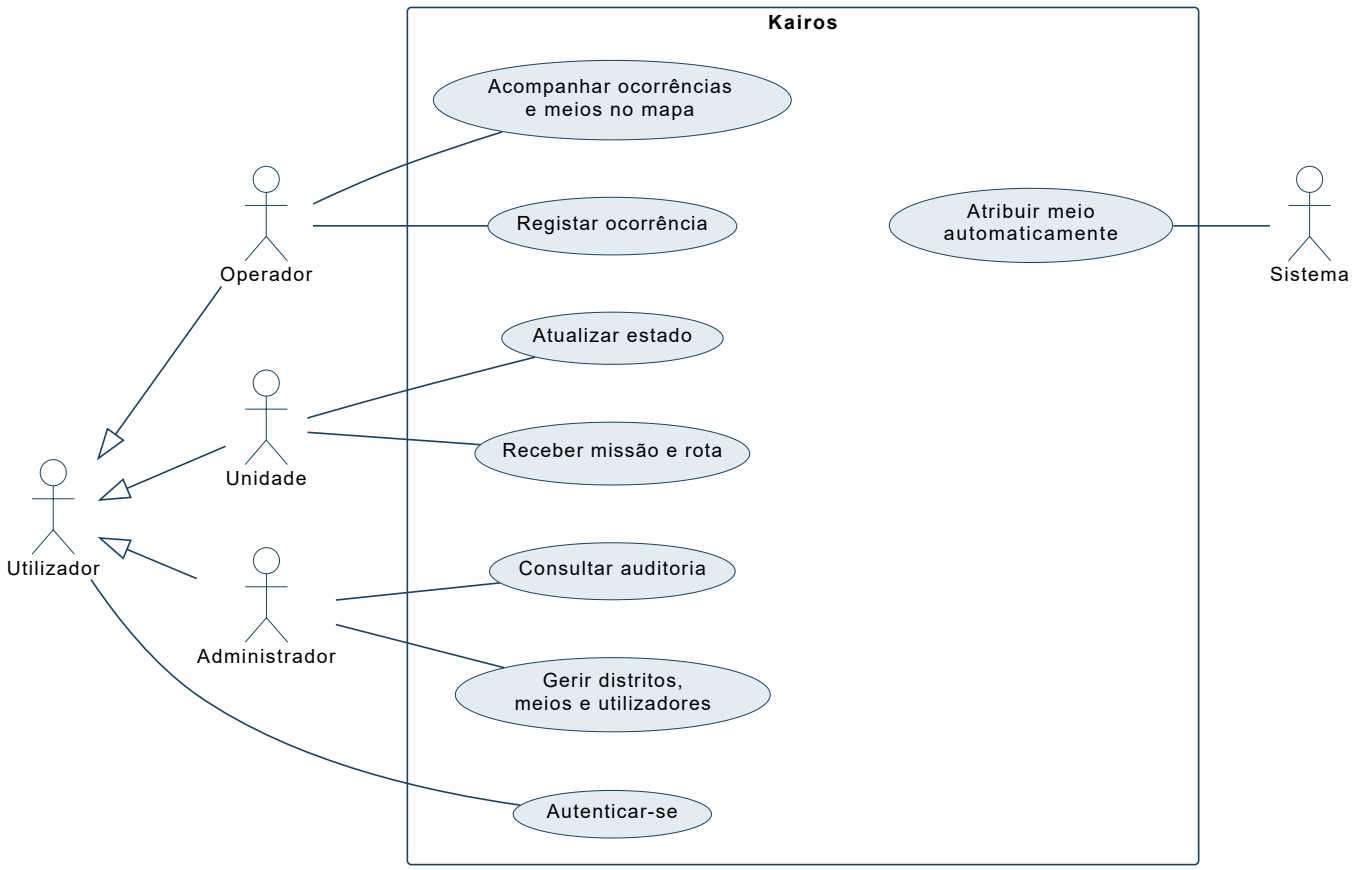
ID	Requisito
RF01	Autenticação de utilizadores com perfis de operador, unidade e administrador
RF02	Registo de ocorrências com tipo, localização, nome da vítima, descrição breve, número de vítimas e **nível de gravidade** (ver 7.5)
RF03	Visualização das ocorrências e unidades do distrito num mapa em tempo real
RF04	Proposta automática de atribuição de unidades (CSP), com confirmação ou rejeição pelo operador

ID	Requisito
RF05	Reatribuição de unidades em trânsito quando surge uma ocorrência de gravidade superior
RF06	Pedido de apoio a distritos vizinhos quando o distrito não tem unidades adequadas
RF07	Cálculo e apresentação da rota mais rápida até à ocorrência (A*)
RF08	Vista da unidade: receber missão, ver rota e atualizar estado
RF09	Atualização periódica da posição das unidades
RF10	Gestão de unidades e utilizadores (administrador)
RF11	Registo de auditoria de todas as atribuições e informações relevantes

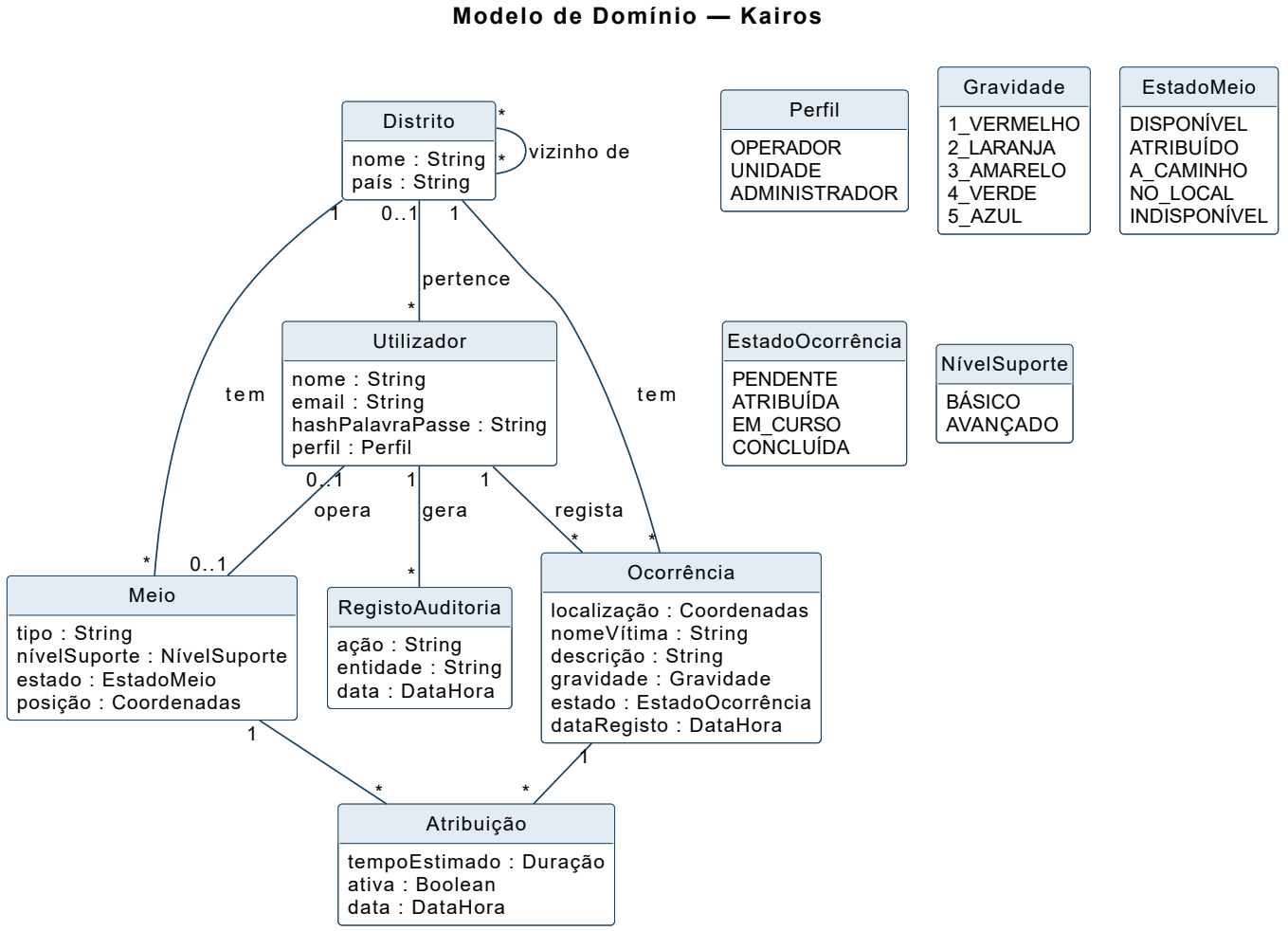
7.2 Requisitos Não Funcionais

ID	Categoria	Requisito
RNF01	Desempenho	A proposta de atribuição é apresentada em menos de 2 segundos após o registo da ocorrência
RNF02	Tempo real	Alterações de estado chegam aos utilizadores interessados em menos de 1 segundo
RNF03	Recuperação	Após a falha de um componente, a redundância é reposta automaticamente
RNF04	Rastreabilidade	Todas as decisões de atribuição ficam registadas com autor e data
RNF05	Usabilidade	Interface responsiva, utilizável em computador e no telemóvel da unidade
RNF06	Disponibilidade	O serviço mantém-se disponível perante a falha de qualquer componente individual ou de uma máquina do cluster
RNF07	Integridade	Nenhuma ocorrência confirmada é perdida
RNF08	Segurança	Toda a comunicação externa é cifrada e o acesso é controlado por perfil
RNF09	Privacidade	Os dados pessoais das vítimas são cifrados na base de dados e só são visíveis a quem trata a ocorrência

7.3 Casos de Uso



7.4 Modelo de Domínio



7.5 Escala de Gravidade

A gravidade de cada ocorrência é atribuída pelo operador numa escala de cinco níveis, inspirada na **Triagem de Manchester** utilizada nos serviços de urgência hospitalar em Portugal. O tipo de ocorrência sugere um nível inicial, que o operador pode alterar.

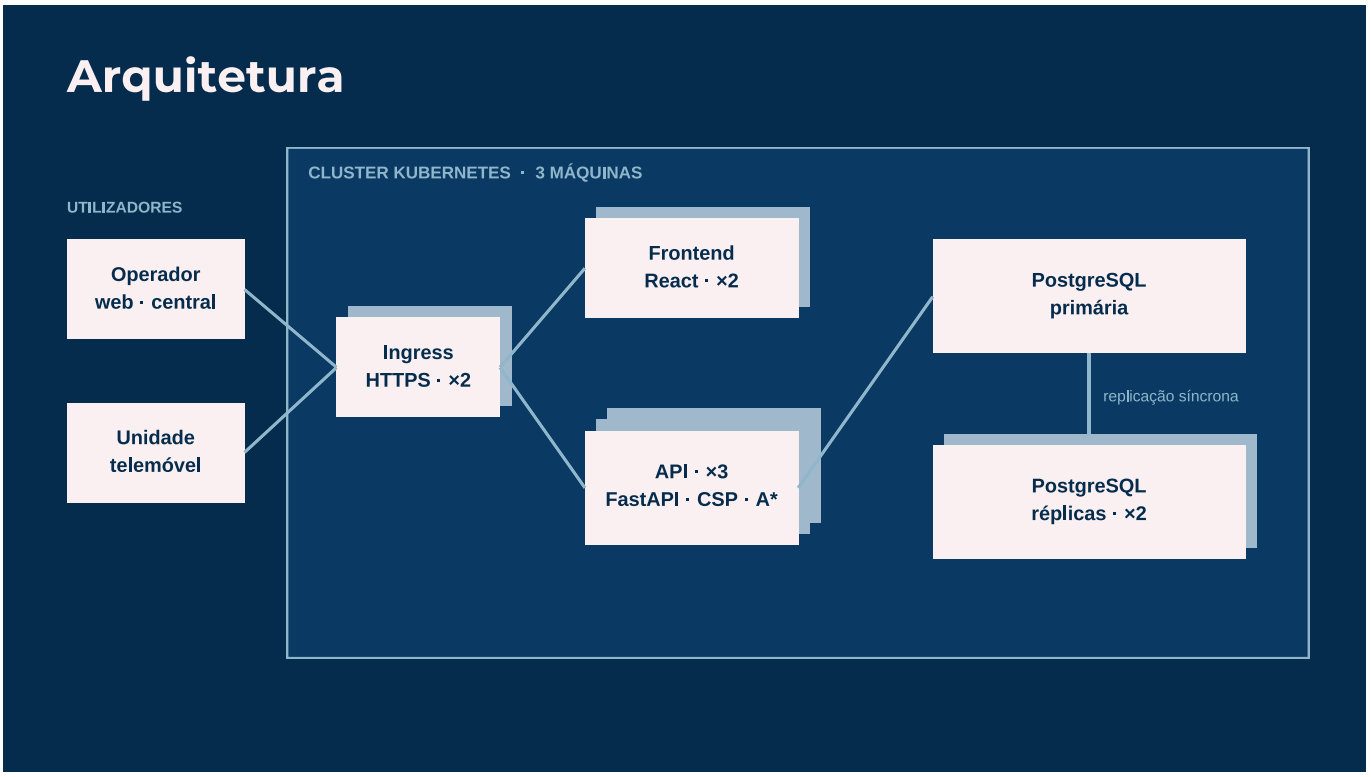
Nível	Cor	Exemplos	Tempo-alvo de chegada	Peso no CSP
1 – Emergente	Vermelho	Paragem cardiorrespiratória, vítima inconsciente	8 min	16
2 – Muito urgente	Laranja	Dor torácica, suspeita de AVC, hemorragia grave	15 min	8
3 – Urgente	Amarelo	Fratura exposta, queimadura moderada	30 min	4
4 – Pouco urgente	Verde	Fratura fechada, entorse, ferida ligeira	60 min	2
5 – Não urgente	Azul	Transporte não urgente, queixa ligeira	120 min	1

Os tempos-alvo e os pesos são **definidos pelo grupo para este projeto**. Os tempos da Triagem de Manchester referem-se à observação médica no hospital e não à chegada de meios ao local.

---

8. Arquitetura Distribuída

8.1 Visão Geral



O sistema corre num cluster **Kubernetes** com **três nós**, cada um numa máquina virtual distinta. Todos os nós executam serviços aplicacionais, e os componentes replicados são distribuídos por nós diferentes, de forma a que a perda de uma máquina não elimine todas as réplicas de um componente. Os três nós são servidores k3s com etcd embutido, pelo que a perda de um nó não compromete a gestão do cluster.

8.2 Componentes

Camada	Componente	Função	Mecanismo de redundância
Entrada	**Ingress (Traefik)**	Ponto de entrada HTTPS, encaminha pedidos para o frontend e a API	2 réplicas em nós diferentes. O serviço é exposto em todos os nós, pelo que qualquer nó aceita pedidos
Cliente	**Frontend** (React + Leaflet)	Interface web responsiva	`Deployment` com 2 réplicas
Serviços	**API** (FastAPI)	Autenticação, ocorrências, rotas (A*), WebSockets	`Deployment` stateless com 3 réplicas, o `Service` distribui os pedidos
Dados	**PostgreSQL**	Ocorrências, unidades, atribuições, utilizadores, auditoria	**CloudNativePG** com 3 instâncias (1 primária + 2 réplicas, uma por nó): replicação em streaming, réplica síncrona e failover automático
Testes	**Simulador**	Gera ocorrências e movimento de unidades	---

8.3 Fluxo Principal

#	Passo
1	O operador preenche o formulário da ocorrência e o pedido chega a qualquer réplica da API
2	A API grava a ocorrência com o estado pendente
3	A API corre o CSP sobre as ocorrências pendentes do distrito e os meios disponíveis
4	A API grava a atribuição e responde ao operador com o meio atribuído
5	A unidade atribuída e os operadores do distrito recebem a atualização por WebSocket

A BD funciona simultaneamente como armazenamento e como fila de trabalho: uma ocorrência só sai do estado *pendente* quando a sua atribuição é gravada. Assim, uma falha a meio do processamento não perde a ocorrência, que continua pendente e é retomada.

8.4 Decisões Arquiteturais

Decisão	Justificação
Kubernetes	Recria automaticamente componentes que falham, distribui réplicas por várias máquinas e oferece deteção de falhas

Decisão	Justificação
API stateless com autenticação **JWT**	Qualquer instância atende qualquer pedido, a falha de uma instância não termina sessões
Base de dados replicada	Todas as garantias necessárias são asseguradas pela BD
Réplica síncrona no PostgreSQL	Uma transação só é confirmada depois de existir também na réplica, pelo que o failover não perde dados confirmados
Consistência forte nas atribuições	O CSP decide a atribuição segundo os critérios de desempate (11.1), mas é a BD que garante que um meio não é atribuído duas vezes. Se duas réplicas da API tentarem atribuir o mesmo meio ao mesmo tempo, só uma escrita tem sucesso, a outra réplica volta a correr o CSP com o estado atualizado

9. Modelo de Faltas

Questão	Resposta
Componentes que podem falhar	Pods da API, do frontend e do ingress. Primária ou réplica do PostgreSQL. Uma máquina inteira do cluster. Ligação de rede entre componentes ou de um cliente
Tipos de falha considerados	Falha por paragem (*crash*): o componente deixa de responder. Interrupção temporária de comunicação.
Falhas simultâneas suportadas	A falha de **uma máquina do cluster** (e de todos os pods que nela correm)
Funcionalidades disponíveis após a falha	Todas: registo de ocorrências, atribuição, rotas, mapa em tempo real e vista das unidades
Deteção da falha	*Probes* do Kubernetes nos pods. Estado *NotReady* dos nós. Monitorização da primária pelo CloudNativePG
Recuperação ou substituição	O Kubernetes recria os pods em falha, noutro nó se necessário, o tráfego deixa de ser enviado para pods em falha, o CloudNativePG promove a réplica a primária
Informação que pode ser perdida	Nenhuma ocorrência ou atribuição **confirmada**. Um pedido em curso no momento da falha pode falhar e o operador repete-o. Avisos em tempo real emitidos durante a falha podem perder-se, mas o cliente recupera o estado atual ao religar. A última posição de um meio pode perder-se, sendo substituída pela seguinte
Consistência entre réplicas	Réplica síncrona do PostgreSQL

10. Cenários de Demonstração de Falhas

#	O que falha	Deteção	O que mantém o serviço	Impacto	Perda de dados	Recuperação e reposição
F1	Pod da API	*Probe* do Kubernetes	As outras 2 réplicas recebem os pedidos	Os WebSockets dessa réplica religam automaticamente a outra	Nenhuma	Pod recriado, volta a 3 réplicas
F2	Pod da API a meio de um registo	*Probe* do Kubernetes	A ocorrência fica pendente e é atribuída na próxima execução do CSP	Atribuição atrasa até 30 s	Nenhuma	Pod recriado
F3	PostgreSQL primária	CloudNativePG	Réplica promovida a primária	Escritas falham durante alguns segundos	Nenhuma confirmada	Antiga primária volta como réplica
F4	Máquina inteira	Nó passa a *NotReady*	Réplicas nos outros nós continuam a servir	Breve degradação	Nenhuma confirmada	Pods recriados nos outros nós
F5	Ligação de rede de uma unidade	Erro no envio	A unidade guarda o estado localmente	Posição desatualizada temporariamente	Nenhuma	Reenvia ao recuperar a ligação

Além das falhas, é demonstrado o cenário de **concorrência**: duas ocorrências registadas ao mesmo tempo, em réplicas diferentes, que disputam o mesmo meio. Só uma fica com ele e a outra recebe a melhor alternativa.

---

11. Componente de Inteligência Artificial

A IA resolve dois problemas.

11.1 Atribuição de Meios - CSP

Sempre que é registada uma ocorrência, e periodicamente a cada 30 segundos, a API corre o CSP sobre as ocorrências pendentes desse distrito. A execução periódica garante que uma ocorrência que ficou pendente por falha de uma réplica é retomada.

Elemento	Descrição
Variáveis	Uma por meio necessário em cada ocorrência pendente

Elemento	Descrição
Domínios	Meios do tipo certo disponíveis ou a caminho de uma ocorrência menos grave, se não houver nenhum, meios dos distritos vizinhos
Restrições obrigatórias	Um meio por ocorrência, tipo de meio adequado, suporte avançado para os níveis 1 e 2, desvio só para uma ocorrência mais grave
Custo a minimizar	Soma dos tempos de chegada × peso da gravidade
Resolução	Backtracking com MRV e *branch and bound*

Desempate quando duas ocorrências querem o mesmo meio:

Ordem	Critério
1	**Gravidade**: a mais grave fica com o meio
2	**Quem perde mais sem ele**: o meio vai para a ocorrência cuja alternativa é pior (resulta do custo total do CSP)
3	**Antiguidade**: a ocorrência registada primeiro

11.2 Cálculo de Rotas - A*

Elemento	Descrição
Grafo	Rede de estradas dos distritos da demonstração, obtida do OpenStreetMap
Custo	Tempo de viagem de cada troço (comprimento ÷ velocidade máxima)
Heurística	Distância em linha reta ÷ velocidade máxima da rede
Utilização	Tempos de chegada para o CSP e rota apresentada no mapa

12. Segurança

Área	Medida
Autenticação	Palavras-passe com *hash* (bcrypt), tokens **JWT** com validade curta
Autorização	Acesso por perfil e por distrito: o operador só vê o seu distrito, a unidade só vê a sua missão
Dados pessoais	Nome da vítima cifrado na BD, recolhem-se apenas os dados necessários (RGPD)
Comunicação	HTTPS e WebSockets cifrados no ingress
Segredos	Credenciais em *Secrets* do Kubernetes, nunca no repositório
Auditoria	Registo de todas as atribuições e alterações, com utilizador e data
Proteção da API	Validação dos dados de entrada

13. Tecnologias a Utilizar

Camada	Tecnologias
Frontend	React, Leaflet, OpenStreetMap, WebSockets
Backend	Python, FastAPI
Inteligência Artificial	Python, OSMnx (obtenção do grafo de estradas)
Base de Dados	PostgreSQL, CloudNativePG
Infraestrutura	Docker, Kubernetes (K3s), Traefik
Ferramentas	GitHub, Trello, VSCode, Postman, Discord

---

14. Guiões de Teste

Guião 1 - Registar ocorrência (Core)

1. O operador do distrito de Lisboa autentica-se.
2. Abre o formulário de nova ocorrência e preenche: localização no mapa, nome da vítima, descrição breve e nível 1 (vermelho).
3. O sistema atribui automaticamente a ambulância de suporte avançado com menor tempo de chegada.
4. A unidade recebe a missão e a rota de imediato, e a atribuição aparece no mapa do operador.

Resultado esperado: O meio é atribuído em menos de 2 segundos.

Guião 2 - Reatribuição por gravidade

1. Uma ambulância está a caminho de uma ocorrência de nível 4 (verde).
2. É registada uma ocorrência de nível 1 próxima, sem outro meio adequado disponível.
3. A ambulância é desviada para a ocorrência de nível 1.

Resultado esperado: A ocorrência de nível 4 volta a ficar pendente e recebe outro meio quando houver.

Guião 3 - Ocorrências simultâneas

1. Dois operadores registam ao mesmo tempo ocorrências próximas, com apenas uma ambulância disponível por perto.
2. Os pedidos chegam a réplicas diferentes da API.

Resultado esperado: A ambulância é atribuída a uma só ocorrência, segundo os critérios de desempate, a outra recebe a melhor alternativa.

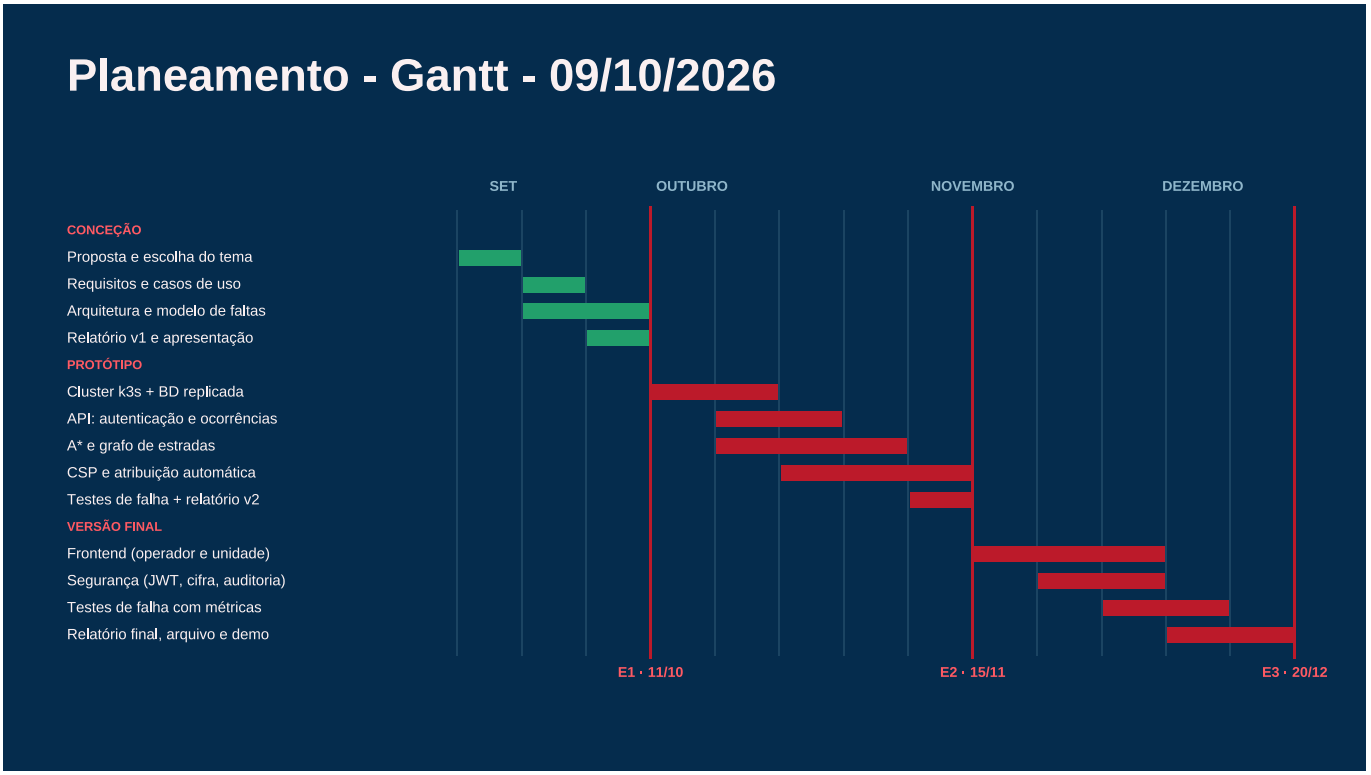
---

15. Planeamento

15.1 Metas por Marco

Marco	Metas internas
Entrega 1 – até 11/10	Conceção, requisitos, arquitetura, modelo de faltas, relatório v1
Entrega 2 – até 15/11	Cluster k3s com PostgreSQL replicado, API com registo de ocorrências, A* e CSP, simulador, primeiros testes de falha, relatório v2
Entrega 3 – até 20/12	Frontend completo, segurança, testes de falha com métricas, demonstração final, arquivo documental

15.2 Gráfico de Gantt



15.3 Riscos

Risco	Probabilidade	Impacto	Mitigação
Complexidade do Kubernetes atrasa o projeto	Média	Alto	**Plano alternativo com Docker Compose**, mantendo as mesmas réplicas
Grafo de estradas demasiado grande para o A*	Média	Médio	Limitar a região da demonstração
CSP lento com muitas ocorrências pendentes	Baixa	Médio	Tempo limite, devolvendo a melhor solução encontrada

16. Conclusão

A **Kairos** pretende demonstrar como uma arquitetura distribuída e tolerante a faltas, combinada com algoritmos de Inteligência Artificial, pode suportar um sistema crítico como uma central de despacho de emergências.

Nesta primeira fase foram definidos o problema, os requisitos, a arquitetura, o modelo de faltas e os cenários de falha a demonstrar. Na **2.ª entrega** será desenvolvido um protótipo funcional com:

- Cluster Kubernetes com PostgreSQL replicado
- API replicada com registo e atribuição automática de ocorrências
- Implementação do A* e do CSP
- Primeiros testes de falha

---

17. Bibliografia

[1] Kubernetes Documentation – https://kubernetes.io/docs
[2] K3s Documentation – https://docs.k3s.io
[3] PostgreSQL Documentation – https://www.postgresql.org/docs
[4] CloudNativePG Documentation – https://cloudnative-pg.io/documentation
[5] FastAPI Documentation – https://fastapi.tiangolo.com
[6] OpenStreetMap – https://www.openstreetmap.org
[7] Leaflet – https://leafletjs.com
[8] Boeing, G. (2017). *OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks*. Computers, Environment and Urban Systems, 65, 126–139.
[9] Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.ª ed.). Pearson.
[10] Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2011). *Distributed Systems: Concepts and Design* (5.ª ed.). Addison-Wesley.
[11] Mackway-Jones, K., Marsden, J., & Windle, J. (Eds.). (2014). *Emergency Triage: Manchester Triage Group* (3.ª ed.). Wiley-Blackwell.
[12] INEM – Instituto Nacional de Emergência Médica – https://www.inem.pt